

Lab Assignment-9.4

2303a51355

Batch-10

PROMPT

Create and Add Google-style docstrings to the given Python functions. Include simple explanation, parameters with types, return values, and one example for each function. Make it clear and easy to understand.

CODE :def add_numbers(a, b):

"""

Adds two numbers and returns the result.

Args:

a (float): The first number.

b (float): The second number.

Returns:

float: The sum of the two numbers.

Example:

```
>>> add_numbers(4, 6)
```

```
10.0
```

```
"""    return a
```

```
+ b
```

def divide_numbers(a, b):

"""

Divides one number by another.

Args:

a (float): The numerator.

b (float): The denominator.

Returns:

float: The result of the division.

Raises:

ValueError: If the denominator is zero.

Example:

```
>>> divide_numbers(10, 2)
5.0
""" if b
== 0:
    raise ValueError("Cannot divide by zero")
return a / b
```

def is_even(number):

"""

Checks whether a given number is even.

Args:

number (int): The number to check.

Returns:

bool: True if the number is even, otherwise False.

Example:

```
>>> is_even(8)
True
```

```

"""

return number % 2 == 0

def reverse_string(text):
    """
    Reverses the given string.

    Args:
        text (str): The string to reverse.

    Returns:
        str: The reversed string.

    Example:
        >>> reverse_string("AI")
        'IA'    """
    return text[::-1]

if __name__ == "__main__":
    print("===== U lity Func ons Program =====")

    try:
        # Input for arithme c opera ons
        num1 =
float(input("Enter first number: "))    num2 =
float(input("Enter second number: "))

        addi on_result = add_numbers(num1, num2)
    print("Addi on Result:", addi on_result)

```

```

division_result = divide_numbers(num1, num2)

print("Division Result:", division_result)


# Input for even check

number = int(input("Enter a number to check even or odd: "))

print("Is Even:", is_even(number))


# Input for string reversal

text = input("Enter a string to reverse: ")

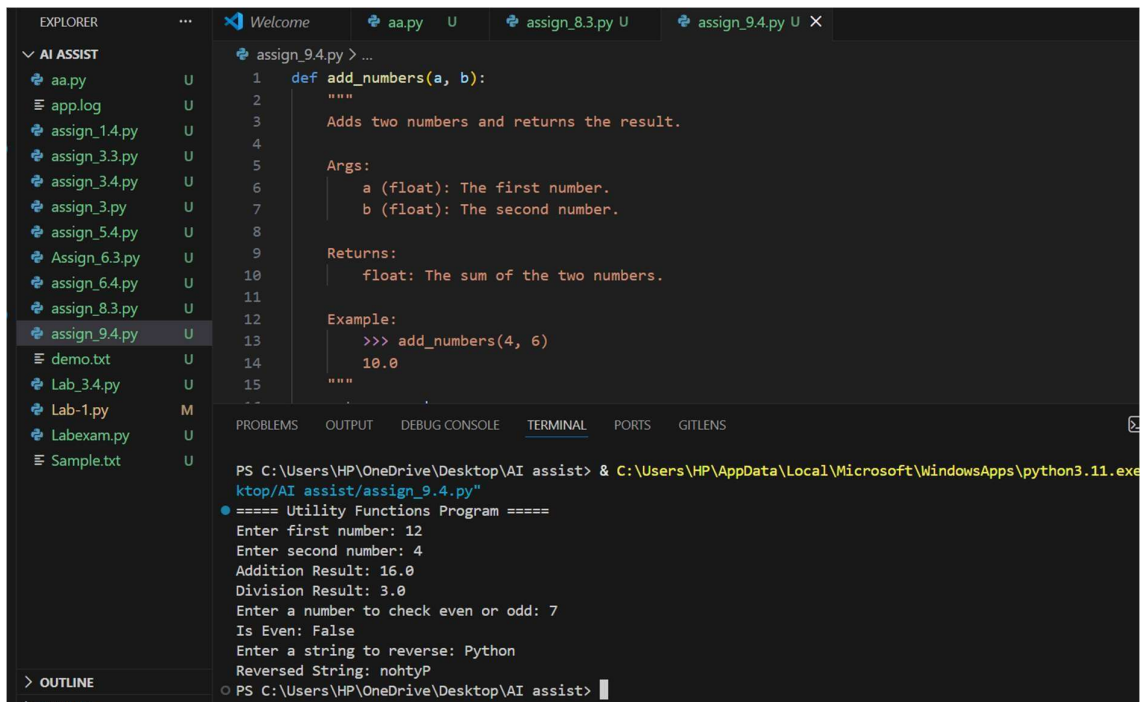
print("Reversed String:", reverse_string(text))

except ValueError as error:

    print("Error:", error)

```

OUTPUT



The screenshot shows a VS Code editor with a file named `assign_9.4.py` open. The file contains a function `add_numbers(a, b)` with a detailed docstring explaining its purpose, arguments, and return value. Below the function, there is an example of how to use it. The terminal window at the bottom shows the command to run the script and the resulting output, which includes the program's execution flow and the results of the function calls.

```

1  def add_numbers(a, b):
2      """
3      Adds two numbers and returns the result.
4
5      Args:
6          a (float): The first number.
7          b (float): The second number.
8
9      Returns:
10         float: The sum of the two numbers.
11
12     Example:
13         >>> add_numbers(4, 6)
14         10.0
15     """

```

```

PS C:\Users\HP\OneDrive\Desktop\AI assist> & C:\Users\HP\AppData\Local\Microsoft\WindowsApps\python3.11.exe
ktop/AI assist/assign_9.4.py"
• ===== Utility Functions Program =====
Enter first number: 12
Enter second number: 4
Addition Result: 16.0
Division Result: 3.0
Enter a number to check even or odd: 7
Is Even: False
Enter a string to reverse: Python
Reversed String: nohtyP
PS C:\Users\HP\OneDrive\Desktop\AI assist>

```

ANALYSIS

The AI added proper docstrings to all func ons in a clear format. It explained what each func on does and how to use it with examples. This makes the code easier for new students or developers to understand.

Task 2: Enhancing Readability Through AI-Generated Inline Comments

PROMPT

Insert concise inline comments into the given Python script to clarify complex or nonobvious logic such as algorithm steps, loop reasoning, optimizations, and conditional decisions. The comments should explain why the logic is used rather than what basic Python syntax does, and they should improve readability without cluttering the code.

CODE

```
def fibonacci(n):    if
n <= 0:        return []
if n == 1:        return
[0]    sequence = [0,
1]    for _ in
range(2, n):
        sequence.append(sequence[-1] + sequence[-
2])    return sequence def bubble_sort(arr):    n =
len(arr)    for i in range(n):        swapped = False

        for j in range(0, n - i - 1):            if
arr[j] > arr[j + 1]:                arr[j], arr[j
+ 1] = arr[j + 1], arr[j]
swapped = True

        if not swapped:
            break

    return arr def
binary_search(arr, target):
    le , right = 0, len(arr) - 1

    while le <= right:
        mid = (le + right) // 2
```

```

        if arr[mid] == target:
            return mid

        elif arr[mid] < target:
            le = mid + 1

    else:
        right = mid - 1

    return -1 if __name__ ==
    "__main__":
        print("==== Algorithm Program =====")

        n = int(input("Enter number of Fibonacci terms: "))
        print("Fibonacci Sequence:", fibonacci(n))    numbers_input =
        input("Enter numbers separated by space: ")    numbers =
        list(map(int, numbers_input.split()))    sorted_list =
        bubble_sort(numbers)    print("Sorted List:", sorted_list)

        target = int(input("Enter number to search: "))
        result = binary_search(sorted_list, target)

        if result != -1:
            print("Element found at index:", result)
        else:
            print("Element not found")

```

OUTPUT

```

1 def fibonacci(n):
2     if n <= 0:
3         return []
4
5     if n == 1:
6         return [0]
7
8     sequence = [0, 1]
9
10    # Generate next values using sum of last two elements
11    for i in range(2, n):

```

```

PS C:\Users\HP\OneDrive\Desktop\AI assist> & C:\Users\HP\AppData\Local\Microsoft\WindowsApps\python3.11.exe "c:/Users/HP/OneDrive/Desktop/AI assist/assign_9.4.py"
==== Algorithm Program ====
Enter number of Fibonacci terms: 6
Fibonacci Sequence: [0, 1, 1, 2, 3, 5]
Enter numbers separated by space: 9 4 7 2 8
Sorted List: [2, 4, 7, 8, 9]
Enter number to search: 7
Element found at index: 2
PS C:\Users\HP\OneDrive\Desktop\AI assist> & C:\Users\HP\AppData\Local\Microsoft\WindowsApps\python3.11.exe "c:/Users/HP/OneDrive/Desktop/AI assist/assign_9.4.py"
==== Algorithm Program ====
Enter number of Fibonacci terms: 5
Fibonacci Sequence: [0, 1, 1, 2, 3]
Enter numbers separated by space: 10 3 6 1
Sorted List: [1, 3, 6, 10]
Enter number to search: 8
Element not found
PS C:\Users\HP\OneDrive\Desktop\AI assist> & C:\Users\HP\AppData\Local\Microsoft\WindowsApps\python3.11.exe "c:/Users/HP/OneDrive/Desktop/AI assist/assign_9.4.py"
==== Algorithm Program ====
Enter number of Fibonacci terms: 1
Fibonacci Sequence: [0]
Enter numbers separated by space: 5 2
Sorted List: [2, 5]
Enter number to search: 5
Element found at index: 1

```

ANALYSIS

The AI added clear and concise inline comments only where the logic was complex. It explained why certain steps like early stopping in sorting and midpoint calculation in binary search are used. This improved readability without making the code look cluttered.

Task 3: Generating Module-Level Documentation for a Python Package

PROMPT

Write a professional multi-line module-level docstring for the given Python module. The docstring should clearly describe the purpose of the module, required dependencies, key classes and functions, and include a short example of how the module can be used. Ensure the tone is suitable for an internal repository or real-world project.

CODE

```
class Student:
    def __init__(self, student_id, name, marks):
```

```
        self.student_id =
```

```
        student_id
        self.name = name
```

```
        self.marks = marks
```

```
    def calculate_average(self):
```

```
        if not self.marks:
```

```
            return 0
```

```

        return round(sum(self.marks) / len(self.marks), 2)

def add_student(student_list, student):
    student_list.append(student)

def find_student_by_id(student_list, student_id):
    for student in student_list:
        if student.student_id == student_id:
            return student
    return None

if __name__ == "__main__":
    print("===== Student Management Demo =====")

    students = []

    sid = int(input("Enter Student ID: "))
    name = input("Enter Student Name: ")
    marks_input = input("Enter marks separated by space: ")
    marks = list(map(int, marks_input.split()))

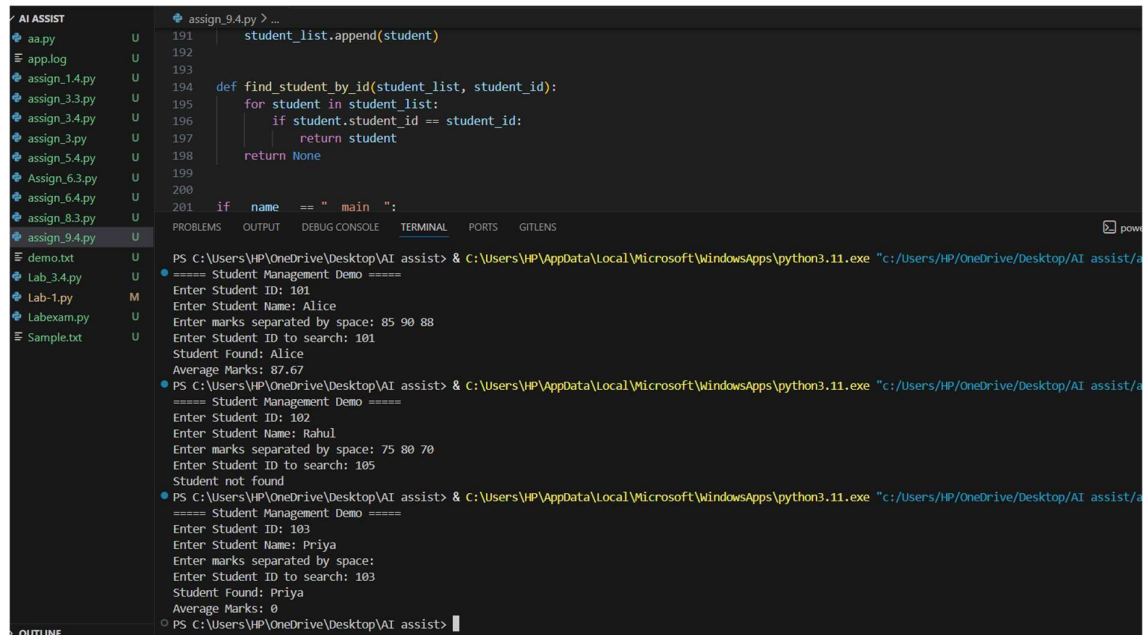
    student = Student(sid, name, marks)
    add_student(students, student)

    search_id = int(input("Enter Student ID to search: "))
    result = find_student_by_id(students, search_id)

    if result:
        print("Student Found:", result.name)
        print("Average Marks:", result.calculate_average())
    else:
        print("Student not found")

```


OUTPUT



```
AI ASSIST
aa.py
app.log
assign_1.4.py
assign_3.3.py
assign_3.4.py
assign_3.py
assign_5.4.py
Assign_6.3.py
assign_6.4.py
assign_8.3.py
assign_9.4.py
demo.txt
Lab_3.4.py
Lab-1.py
Labexam.py
Sample.txt

assign_9.4.py > ...
191     student_list.append(student)
192
193
194 def find_student_by_id(student_list, student_id):
195     for student in student_list:
196         if student.student_id == student_id:
197             return student
198     return None
199
200
201 if __name__ == "__main__":
202     # Student Management Demo
203     Enter Student ID: 101
204     Enter Student Name: Alice
205     Enter marks separated by space: 85 90 88
206     Enter Student ID to search: 101
207     Student Found: Alice
208     Average Marks: 87.67
209
210     # Student Management Demo
211     Enter Student ID: 102
212     Enter Student Name: Rahul
213     Enter marks separated by space: 75 80 70
214     Enter Student ID to search: 105
215     Student not found
216
217     # Student Management Demo
218     Enter Student ID: 103
219     Enter Student Name: Priya
220     Enter marks separated by space:
221     Enter Student ID to search: 103
222     Student Found: Priya
223     Average Marks: 0
224
225 PS C:\Users\HP\OneDrive\Desktop\AI assist> |
```

ANALYSIS

The AI generated a clear and well-structured module-level docstring at the top of the file. It explained the purpose, dependencies, and key components in a professional way. This makes the module easy to understand for anyone who opens the file.

Task 4: Converting Developer Comments into Structured Docstrings

PROMPT

Convert the existing inline developer comments inside the functions into structured Google-style docstrings. Preserve the original meaning of the comments, include description, parameters, return values, and an example where appropriate. Remove redundant inline comments and convert them into docstrings to keep the function bodies clean.

CODE

```
def calculate_discount(price, discount_percentage):
```

```
    """
```

Calculate the final price after applying a discount.

Args:

price (float): The original price of the product.

discount_percentage (float): The percentage of discount to apply.

Returns:

float: The final discounted price.

Returns 0 if the price is negative.

Example:

```
>>> calculate_discount(200, 10)
```

```
180.0
```

```
""" if
```

```
price < 0:
```

```
return 0
```

```
discount = (price * discount_percentage) /
```

```
100 final_price = price - discount return
```

```
final_price
```

```
def find_max(numbers):
```

```
"""
```

```
Find the maximum value in a list of numbers.
```

Args:

numbers (list of int or float): List of numbers to evaluate.

Returns:

int or float: The largest number in the list.

Returns None if the list is empty.

Example:

```
>>> find_max([5, 9, 3])
```

```
9
```

```

"""

if not numbers:

    return None


maximum =
numbers[0]    for num in
numbers:      if num >
maximum:
maximum = num


return maximum if
__name__ == "__main__":
    print("===== Discount Calculator & Maximum Finder =====")


    # User input for discount calcula on
price = float(input("Enter original price: "))
    discount_percentage = float(input("Enter discount percentage: "))


    final_price = calculate_discount(price, discount_percentage)
print("Final Price a er Discount:", final_price)


    # User input for maximum finder
    numbers_input = input("Enter numbers separated by space: ")
numbers = list(map(float, numbers_input.split()))

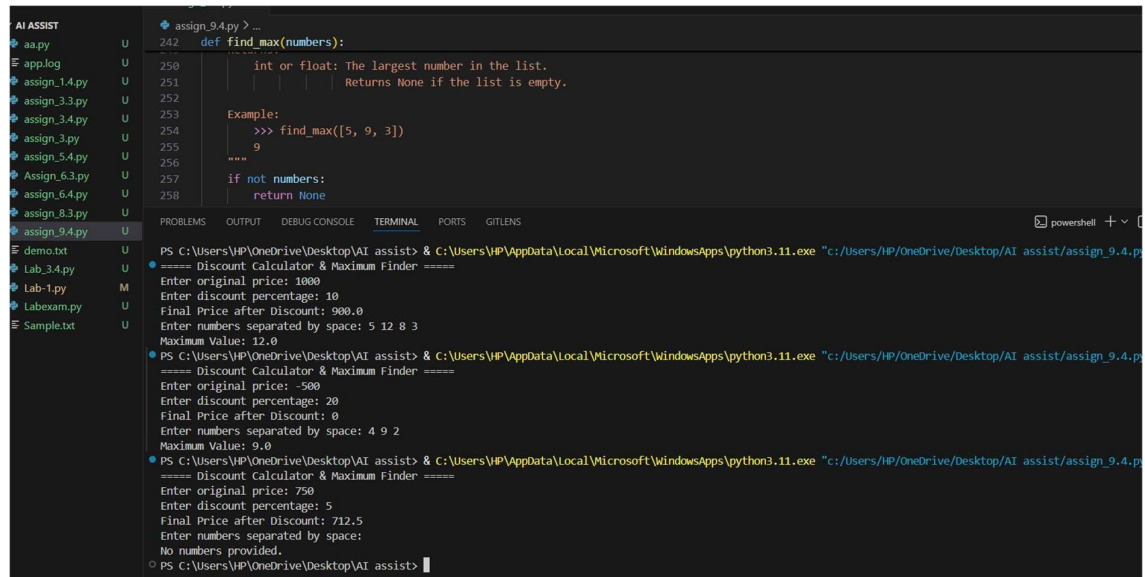

    max_value = find_max(numbers)


    if max_value is not None:
        print("Maximum Value:", max_value)
    else:

```

```
print("No numbers provided.")
```

OUTPUT



```
AI ASSIST
aa.py U
app.log U
assign_1.4.py U
assign_3.3.py U
assign_3.4.py U
assign_3.py U
assign_5.4.py U
Assign_6.3.py U
assign_6.4.py U
assign_8.3.py U
assign_9.4.py U
demo.txt U
Lab_3.4.py U
Lab-1.py M
Labexam.py U
Sample.txt U

def find_max(numbers):
    """
    int or float: The largest number in the list.
    Returns None if the list is empty.

    Example:
    >>> find_max([5, 9, 3])
    9
    """
    if not numbers:
        return None

PS C:\Users\HP\OneDrive\Desktop\AI assist> & C:\Users\HP\AppData\Local\Microsoft\WindowsApps\python3.11.exe "c:\Users\HP\OneDrive\Desktop\AI assist\assign_9.4.py"
==== Discount Calculator & Maximum Finder ====
Enter original price: 1000
Enter discount percentage: 10
Final Price after Discount: 900.0
Enter numbers separated by space: 5 12 8 3
Maximum Value: 12.0
PS C:\Users\HP\OneDrive\Desktop\AI assist> & C:\Users\HP\AppData\Local\Microsoft\WindowsApps\python3.11.exe "c:\Users\HP\OneDrive\Desktop\AI assist\assign_9.4.py"
==== Discount Calculator & Maximum Finder ====
Enter original price: -500
Enter discount percentage: 20
Final Price after Discount: 0
Enter numbers separated by space: 4 9 2
Maximum Value: 9.0
PS C:\Users\HP\OneDrive\Desktop\AI assist> & C:\Users\HP\AppData\Local\Microsoft\WindowsApps\python3.11.exe "c:\Users\HP\OneDrive\Desktop\AI assist\assign_9.4.py"
==== Discount Calculator & Maximum Finder ====
Enter original price: 750
Enter discount percentage: 5
Final Price after Discount: 712.5
Enter numbers separated by space:
No numbers provided.
```

ANALYSIS

The AI successfully transformed the long inline comments into proper structured docstrings. It preserved the original explanation while making the code cleaner and more professional. The function bodies became less cluttered, improving readability and consistency across the code.

Task 5: Building a Mini Automatic Documentation Generator

PROMPT

Design a Python utility that reads a given .py file, detects all functions and classes, and automatically inserts placeholder Google-style docstrings for each. The tool should generate documentation scaffolding and save the updated content into a new file without modifying the original

CODE

```
import re
import
```

```
os
```

```
def generate_docstring(name, indent, obj_type="function"):
```

```
    """
```

```
    Generates a Google-style placeholder docstring.
```

```

        """        if obj_type ==
"class":

        doc = f'{indent}""Summary of {name}
class.\n\n'        doc += f'{indent}Descrip on:\n'
doc += f'{indent}  TODO: Add class descrip on.\n'
doc += f'{indent}""'\n'    else:

        doc = f'{indent}""Summary of {name} func on.\n\n'
doc += f'{indent}Args:\n'

        doc += f'{indent}  TODO: Add parameter descrip ons.\n\n'
doc += f'{indent>Returns:\n'

        doc += f'{indent}  TODO: Add return descrip on.\n'
doc += f'{indent}""'\n'


    return doc def
process_file(filename):
    """

    Reads the file and inserts placeholder docstrings
a er func ons and class defini ons.

    """

    if not os.path.exists(filename):
print("File does not exist.")
return

    with open(filename, "r") as file:

        lines = file.readlines()

    new_lines = []
    i = 0

```

```

    while i < len(lines):
line      =      lines[i]
new_lines.append(line)

    # Detect func on
    func_match = re.match(r'^(\s*)def\s+(\w+)\s*\(', line)

    # Detect class
    class_match = re.match(r'^(\s*)class\s+(\w+)\s*[\(:]?', line)
if func_match:
    indent = func_match.group(1) + "    "
    func_name = func_match.group(2)

    # Check if next line already has docstring
if i + 1 < len(lines) and '"""' not in lines[i + 1]:
        doc = generate_docstring(func_name, indent, "func on")
new_lines.append(doc)

    elif class_match:
        indent = class_match.group(1) + "    "
        class_name = class_match.group(2)

        # Check if next line already has docstring
if i + 1 < len(lines) and '"""' not in lines[i + 1]:
            doc = generate_docstring(class_name, indent, "class")
new_lines.append(doc)
        i +=
1

```

```
output_file = "documented_" + filename
```

```
with open(output_file, "w") as file:
```

```
    file.writelines(new_lines)
```

```
    print(f"\nDocumenta on generated successfully!")
```

```
print(f"Updated file saved as: {output_file}") if
```

```
__name__ == "__main__":
```

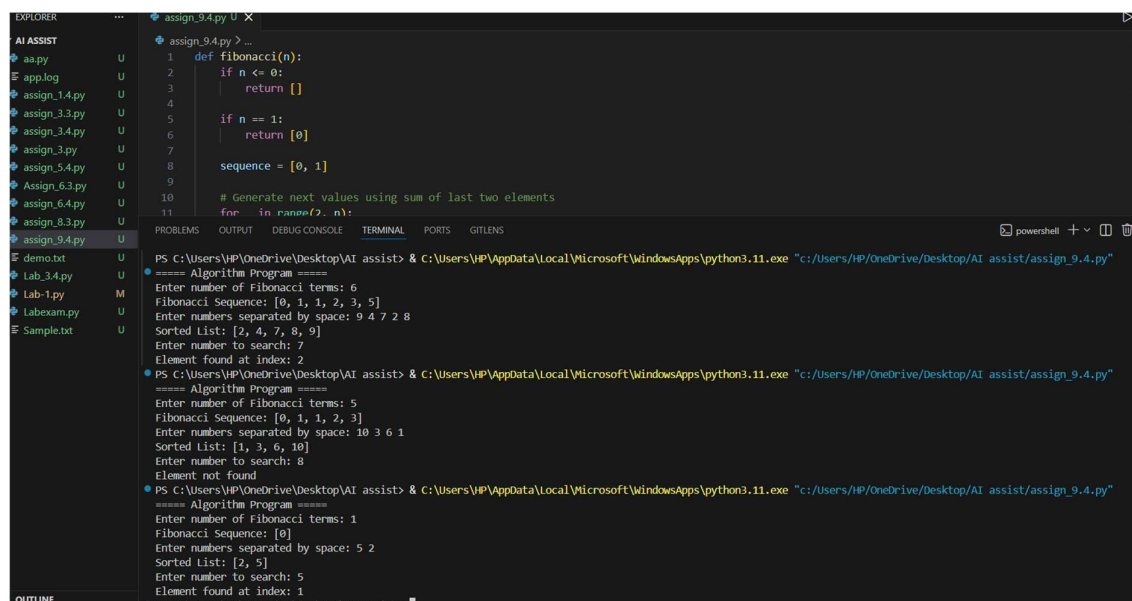
```
    print("Mini Automa c Documenta on Generator")
```

```
    print("-----")
```

```
    user_file = input("Enter the Python file name (with .py extension): ")
```

```
    process_file(user_file)
```

OUTPUT



```
EXPLORER
AI ASSIST
aa.py
app.log
assign_1.4.py
assign_3.3.py
assign_3.4.py
assign_3.py
assign_5.4.py
Assign_6.3.py
assign_6.4.py
assign_8.3.py
assign_9.4.py
demo.txt
Lab_3.4.py
Lab-1.py
Labexam.py
Sample.txt

assign_9.4.py
1 def fibonacci(n):
2     if n <= 0:
3         return []
4
5     if n == 1:
6         return [0]
7
8     sequence = [0, 1]
9
10    # Generate next values using sum of last two elements
11    for i in range(2, n):
12        sequence.append(sequence[-1] + sequence[-2])
13    return sequence
14
15 def process_file(filename):
16     with open(filename, 'r') as file:
17         lines = file.readlines()
18
19         # Extract the number of terms
20         terms = None
21         for line in lines:
22             if line.startswith('Enter number of Fibonacci terms: '):
23                 terms = int(line.split(':')[1].strip())
24                 break
25
26         # Generate the Fibonacci sequence
27         fib_seq = fibonacci(terms)
28
29         # Sort the sequence
30         sorted_seq = sorted(fib_seq)
31
32         # Print the results
33         print(f'Fibonacci Sequence: {fib_seq}')
34         print(f'Sorted List: {sorted_seq}')
35
36         # Prompt for search
37         search = None
38         for line in lines:
39             if line.startswith('Enter number to search: '):
40                 search = int(line.split(':')[1].strip())
41                 break
42
43         # Find the element
44         found = False
45         for i, value in enumerate(sorted_seq):
46             if value == search:
47                 found = True
48                 print(f'Element found at index: {i}')
49                 break
50         if not found:
51             print('Element not found')
52
53     print('==== Algorithm Program =====')
54     print('Enter number of Fibonacci terms: ')
55     print('Fibonacci Sequence: [0, 1, 1, 2, 3, 5]')
56     print('Enter numbers separated by space: 9 4 7 2 8')
57     print('Sorted List: [2, 4, 7, 8, 9]')
58     print('Enter number to search: 7')
59     print('Element found at index: 2')
60
61     print('==== Algorithm Program =====')
62     print('Enter number of Fibonacci terms: 5')
63     print('Fibonacci Sequence: [0, 1, 1, 2, 3]')
64     print('Enter numbers separated by space: 10 3 6 1')
65     print('Sorted List: [1, 3, 6, 10]')
66     print('Enter number to search: 8')
67     print('Element not found')
68
69     print('==== Algorithm Program =====')
70     print('Enter number of Fibonacci terms: 1')
71     print('Fibonacci Sequence: [0]')
72     print('Enter numbers separated by space: 5 2')
73     print('Sorted List: [2, 5]')
74     print('Enter number to search: 5')
75     print('Element found at index: 1')
76
77 if __name__ == '__main__':
78     user_file = input("Enter the Python file name (with .py extension): ")
79     process_file(user_file)
```

ANALYSIS

The solu on involves reading the file, iden fying def and class pa erns using techniques like regular expressions, and inser ng properly indented placeholder docstrings. The main goal is automa on of basic documenta on structure, improving code readability and reducing manual effort for developers.

